

COMPUTE • VM IMPORT

Using virt-v2v on Starlight

Converting foreign-hypervisor VMs — **VMware, Hyper-V, Xen, raw disks** — into KVM/libvirt guests, from inside a Fedora 43 distrobox on the Starlight host.

HOST starlight@192.168.2.82	ENVIRONMENT Fedora 43 · fedora-virt
TOOL VERSION virt-v2v 2.10.0	CAPTURED 2026-06-09 · live host

CONTENTS

01	What virt-v2v does
02	Source / destination matrix
03	Anatomy of a virt-v2v command
04	Bring up the distrobox
05	Converting from VMware
	5.1 ESXi over SSH 5.2 vCenter / VDDK 5.3 Define & boot on Starlight
06	Live walk-through: Fedora 43 cloud image
	6.1 Grab a source disk 6.2 Inspect (dry-run) 6.3 Convert 6.4 Inspect output
	6.5 Boot the result
07	Other recipes
08	Customisation at conversion time
09	Troubleshooting cheat-sheet
10	Pre / post-conversion checklist
11	References



What virt-v2v does

"virt-v2v converts a single guest from a foreign hypervisor to run on KVM."

It reads a VM from a source (vCenter, ESXi, OVA, VMX, raw/qcow2 disk, libvirt XML) and — for Starlight — writes a KVM-ready VM to local libvirt: a directory of disk + domain XML, a libvirt storage pool, or a ready-to-run `qemu` invocation. Along the way it:

- inspects the guest OS,
- installs / configures `virtio` drivers (network, block, balloon, rng),
- removes hypervisor-specific agents (VMware Tools, Hyper-V ICs),
- rewrites bootloader / initrd entries as needed,
- emits a libvirt domain XML matched to the converted disk(s).



Not a live-migration tool.

The source guest must be powered off before conversion.



Source / destination matrix

The shipped binary on Fedora 43 advertises many input/output modes, but on Starlight we only use the local-KVM subset:

● ● ● MODES

SUBSET IN USE

```
input:  disk | libvirt | libvirtxml | ova | vmx
output: disk | libvirt | qemu   (local-only – no oVirt / OpenStack / KubeVirt)
convert: linux | windows
transports: ssh (Xen/ESXi), vddk (vCenter)
```



Anatomy of a virt-v2v command

● ● ● STRUCTURE

```
virt-v2v -i <input-mode> [input-options] <source>  
        -o <output-mode> [output-options]  
        [--customise-options]
```

— KEY FLAGS WORTH MEMORISING

FLAG	PURPOSE
<code>-i disk ova vmx libvirt libvirtxml</code>	Pick the input plug-in
<code>-ic <uri></code>	libvirt connection URI (<code>vpx://</code> , <code>esx://</code> , <code>xen+ssh://</code> , ...)
<code>-it ssh vddk</code>	Transport for ESXi-over-SSH or VMware VDDK
<code>-ip <file></code>	File holding the source-side password
<code>-o disk libvirt qemu</code>	Output plug-in (Starlight uses these only)
<code>-os <storage></code>	Output storage (directory for <code>-o disk</code> , pool name for <code>-o libvirt</code>)
<code>-of qcow2 raw</code>	Output disk format
<code>-on <name></code>	Rename the guest
<code>-b · --network · --mac</code>	Re-map NICs (<code>-b in:out</code> , <code>- network in:out</code> , <code>--mac MAC:network:NAME</code>)
<code>--parallel N</code>	Copy disks in parallel
<code>-m <MB></code>	RAM for the conversion appliance (bump if SELinux relabel OOMs)
<code>--print-source</code>	Read source metadata and exit (great for dry runs)

FLAG	PURPOSE
<code>--in-place</code>	Convert in-place instead of copying (uses <code>virt-v2v-in-place</code>)
<code>--install</code> · <code>--firstboot-command</code> · <code>--password</code> · <code>--hostname</code>	virt-customize-style guest tweaks



SECTION 04

Bring up the distrobox

04

On Starlight (Fedora 43 immutable host):

```

● ● ● SHELL · STARLIGHT HOST BASH

ssh starlight@192.168.2.82

# Single-file fetch – the host has no git installed.
curl -fsSL -o setup-distrobox.sh \
  https://raw.githubusercontent.com/Mainsail-Industries/starlight-
  distrobox/main/setup-distrobox.sh
chmod +x setup-distrobox.sh
./setup-distrobox.sh create --virt --name fedora-virt

```

`--virt` installs `virt-manager` `virt-viewer` `libvirt-client` `virt-v2v` inside the box.

— SANITY CHECK FROM INSIDE THE BOX

```

● ● ● CAPTURED OUTPUT

$ distrobox enter fedora-virt -- bash -c "virt-v2v --version; rpm -q virt-v2v
  libguestfs"
virt-v2v 2.10.0fedora=43,release=1.fc43
virt-v2v-2.10.0-1.fc43.x86_64
libguestfs-1.58.1-1.fc43.x86_64

```

— CAPABILITIES THE BINARY ADVERTISES

● ● ● CAPTURED OUTPUT

--MACHINE-READABLE

```
$ distrobox enter fedora-virt -- virt-v2v --machine-readable
virt-v2v
virt-v2v-2.0
libguestfs-rewrite
vcenter-https
xen-ssh
vddk
colours-option
vdsm-compat-option
io/oo
mac-option
bandwidth-option
mac-ip-option
parallel-option
customize-ops
block-driver-option
input:disk
input:libvirt
input:libvirtxml
input:ova
input:vmx
output:disk
output:glance
output:kubevirt
output:libvirt
output:null
output:openstack
output:ovirt
output:ovirt-upload
output:qemu
output:vdsm
convert:linux
convert:windows
ovf:ovirt
ovf:ovirtexp
```



Converting from VMware

VMware-to-KVM is the headline workflow for virt-v2v, so it gets its own top-level section. Pick one of two paths depending on what you have access to on the source side.

YOU HAVE...	USE	SECTION
Shell on a standalone or unmanaged ESXi host	<code>-i vmx -it ssh</code>	5.1 — ESXi over SSH
vCenter (production, fleet)	<code>-ic vpx://... -it vddk</code>	5.2 — vCenter / VDDK

Both paths end the same way: a qcow2 disk plus a libvirt domain XML inside the distrobox, ready to `virsh define && virsh start` on Starlight (see §5.3).

5.1 ESXi over SSH — simple, vCenter-less

This is the path you reach for when you have **shell on an ESXi host but no vCenter** (lab / edge / standalone ESXi). It is the simplest VMware-side workflow and needs nothing on the ESXi host beyond what ships in the box.

— WHAT IS HAPPENING

- **VMX** is the VM definition file on a VMFS datastore (`/vmfs/volumes/<datastore>/<VM>/<VM>.vmx`). It points at one or more `.vmdk` flat files that hold the actual disk contents. virt-v2v's `-i vmx` parser reads the VMX, finds the VMDKs next to it, and pulls them across the wire.
- The transport `-it ssh` tells virt-v2v to copy those VMDKs by opening an SSH session to the ESXi host and reading the files with `cat` / `dd`. No VMware libraries required.
- Output `-o local -os /var/tmp` writes a libvirt domain XML plus converted disk(s) into `/var/tmp` inside the distrobox — same shape as the live walk-through in §6.4.

— WHY A VSPHERE ADMIN WOULD PICK THIS

- No vCenter licence / vCenter outage / vCenter-less host.
- VDDK (the official VMware copy library) requires download, EULA, and a matching ESXi/vSphere version — SSH skips all of that.
- Great for one-off rescues: "this VM has been off in datastore1 for a year, get it to KVM."

— HOW TO PREPARE THE ESXI HOST

- 1 **Enable SSH on the host.** In the vSphere Web Client / Host Client:
`Host → Manage → Services → TSM-SSH → Start` . Or from a console shell:
`vim-cmd hostsvc/enable_ssh && vim-cmd hostsvc/start_ssh` .
- 2 **Open SSH in the ESXi firewall** if it's restricted:
`esxcli network firewall ruleset set --ruleset-id=sshServer --enabled=true` .
- 3 **Power the VM off** on the source. virt-v2v will refuse a running guest — and you don't want a torn disk image. (Snapshots are fine, but consolidate them first: vSphere Client → VM → Snapshots → *Consolidate.*)
- 4 **Locate the VMX path** in the datastore browser, e.g.
`[datastore1] WebSrv01/WebSrv01.vmx` . Translate to the absolute path under `/vmfs/volumes/` .
- 5 **Stage an SSH key for root@esxi** (the ESXi `root` shell honours `~/.ssh/authorized_keys`). With a password-only host, virt-v2v will prompt; in scripts use `sshpass` or — preferably — drop a key with `ssh-copy-id root@esxi.example.com` .

— HOW TO INVOKE

```
● ● ● SHELL · ESXI OVER SSH BASH

distrobox enter fedora-virt -- \
  virt-v2v -i vmx -it ssh \
    ssh://root@esxi.example.com/vmfs/volumes/datastore1/WebSrv01/WebSrv01.vmx \
    -o local -os /var/tmp
```

What virt-v2v does next, in order:

- 1 SSHes in, reads the VMX, enumerates VMDKs.
- 2 Streams each VMDK over SSH into an overlay on the conversion host.

- 3 Inspects the guest OS, swaps drivers to virtio, removes VMware Tools, rewrites initrd/bootloader as needed.
- 4 Writes the final qcow2 disk and a libvirt domain XML into `/var/tmp`.

— COMMON GOTCHAS

- ♦ **"vmx parser: cannot find disk"** — the VMX references an SE-Sparse or delta VMDK because snapshots weren't consolidated. Consolidate in vSphere first.
- ♦ **Permission denied opening VMDK** — ESXi's `root` shell can read all VMFS files, but a non-root user usually cannot.
- ♦ **Slow copy** — SSH is single-threaded and CPU-bound on the ESXi side. For VMs larger than a few hundred GB, use the vCenter/VDDK path (§5.2) which can read concurrently.

5.2 vCenter / VDDK — production, fast

This is the path for **production fleets managed by vCenter**. You talk to vCenter (not the ESXi host directly), let VMware's own copy library (VDDK) do the heavy lifting, and get parallel / SAN-accelerated disk reads.

— WHAT IS HAPPENING

- ♦ `-ic vpx://...` is a *libvirt connection URI* of the form `vpx://<user>@<vcenter-fqdn>/<Datacenter>/<Cluster>/<ESXi-host>`. It tells virt-v2v "go ask vCenter for this VM's metadata." (`vpx` is vSphere-speak for vCenter's API; `esx://` is the equivalent for a standalone ESXi.)
- ♦ The guest name at the end of the command is the exact VM inventory name as it appears in vCenter — the same string you see in the vSphere Client tree.
- ♦ `-it vddk` switches the disk-read transport to the **VMware Virtual Disk Development Kit**: VMware's official C library for reading VMDKs. With it you get NBD/NBDSSL/SAN/HotAdd modes, parallelism, thin-aware reads, and CBT — exactly the same stack VMware's own backup vendors use.
- ♦ `-io vddk-libdir=...` points at the unpacked VDDK distribution on disk. virt-v2v will `dlopen()` `libvixDiskLib.so` from there.

— WHY A VSPHERE ADMIN WOULD PICK THIS

- vCenter is the source of truth — names, networks, hardware version, snapshots, and folder placement are all there.
- VDDK pulls disks **much** faster than SSH, especially over 10/25 GbE fabrics or directly from SAN LUNs.
- vCenter handles auth (SSO, AD, MFA via a service account) — no per-host root credential sprawl.
- It's the only mode where Changed Block Tracking is meaningful if you later want to redo the conversion against a fresher snapshot.

— HOW TO PREPARE VCENTER / VDDK

- 1 **Create (or pick) a service account** with at minimum:
Provisioning → Allow read-only disk access, Allow disk access, Allow virtual machine download, and Interaction → Power off. Grant it on the Datacenter/Cluster/Folder containing the source VMs. (Plain "Read-only" is *not* sufficient — VDDK needs the download privileges.)
- 2 **Power the VM off (or take a snapshot)**. virt-v2v will refuse a powered-on guest unless you point it at a snapshot.
- 3 **Find the connection URI fragments:** Datacenter (top-level inventory object), Cluster (use host for a standalone host), and ESXi-host (FQDN of the host where the VM is registered).
- 4 **Download and unpack the VDDK** from VMware's developer portal (Broadcom login required). The 8.x release works against modern vCenter:

● ● ● SHELL

```
tar xzf VMware-vix-disklib-8.0.3-23416773.x86_64.tar.gz \  
-C /opt/ # → /opt/vmware-vix-disklib-distrib
```

Make sure the **VDDK major version matches or exceeds** the vCenter / ESXi major version.

- 5 **Stash the password in a file** (virt-v2v refuses passwords on the command line):

● ● ● SHELL

```
printf '%s' 'svc-v2v-password' > /tmp/vpw
chmod 600 /tmp/vpw
```

- 6 **Verify the thumbprint** (HTTPS pin). virt-v2v will print the expected thumbprint on the first connection — capture it and pass via `-io vddk-thumbprint=AA:BB:...` for unattended runs.

— HOW TO INVOKE

● ● ● SHELL · VCENTER / VDDK

BASH

```
distrobox enter fedora-virt -- \
  virt-v2v \
    -ic 'vpx://svc-v2v@vsphere.local@vcenter.example.com/DC1/Cluster01/esxi-
01.example.com?no_verify=1' \
    -ip /tmp/vpw \
    -it vddk \
    -io vddk-libdir=/opt/vmware-vix-disklib-distrib \
    -io vddk-thumbprint=AA:BB:CC:DD:... \
    WebSrv01 \
    -o local -os /var/tmp
```

What virt-v2v does next, in order:

- 1 Authenticates to vCenter, locates `WebSrv01`, reads its OVF/VMX metadata (CPU, RAM, NICs, disks, hardware version, BIOS vs UEFI).
- 2 Spins up VDDK in NBD mode against the ESXi host that owns the VM and streams each VMDK into an overlay locally.
- 3 Same conversion pass as the SSH path: virtio drivers, Tools removal, bootloader fix-ups, SELinux relabel for Linux guests.
- 4 Writes the qcow2 + libvirt domain XML to `/var/tmp`.

— COMMON GOTCHAS

- ♦ **"VDDK library not found"** — wrong `vddk-libdir` path, or the VDDK tarball was extracted incorrectly. The directory must contain `lib64/libvixDiskLib.so`.

- ♦ **"Permission to perform this operation was denied"** — the service account is missing the "Allow virtual machine download" privilege family.
- ♦ **Hostname mismatch / certificate errors** — append `?no_verify=1` to the URI in lab environments, or import the vCenter CA into the distrobox's trust store for production.
- ♦ **VM not found** — the trailing name is **case-sensitive** and must match the inventory name, not the guest hostname or UUID.
- ♦ **Snapshots present** — virt-v2v will convert the **current** state including delta chains; consolidate first unless you specifically want a point-in-time.



ESXi-direct alternative

Swap `vpx://` for `esx://root@esxi-01.example.com/?no_verify=1` to skip vCenter when the host is standalone — VDDK still works.

5.3 Defining and booting the converted VM on Starlight

Both paths above leave you with `/var/tmp/<name>-sda` (qcow2) and `/var/tmp/<name>.xml` (libvirt domain). To register and start on Starlight's libvirt:

● ● ● SHELL · STARLIGHT LIBVIRT

BASH

```
distrobox enter fedora-virt -- bash -c '  
  virsh -c qemu:///system define /var/tmp/<name>.xml  
  virsh -c qemu:///system start <name>  
  virsh -c qemu:///system list --all  
'
```

Open the console with `virt-viewer -c qemu:///system <name>` (or `virt-manager` if you exported the GUI app with `distrobox-export --app virt-manager`).



Convert a Fedora 43 cloud image

The walk-through below uses a plain cloud-image qcow2 rather than a real VMware source, because it (a) needs no external credentials, (b) runs end-to-end in under a minute, and (c) exercises the exact same conversion pass that runs on the VMware paths above.

6.1 Grab a source disk

We use the official Fedora 43 Cloud Base image as the "foreign" VM (in this case the only thing being converted is the bus/driver layout and libvirt metadata — the guest is already Linux).

● ● ● SHELL

BASH

```
distrobox enter fedora-virt -- bash -c '  
  cd /var/tmp  
  curl -sL -o fedora43-cloud.qcow2 \  
    https://download-  
ib01.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-  
Base-Generic-43-1.6.x86_64.qcow2  
  qemu-img info fedora43-cloud.qcow2 | head -10  
'
```

Captured output:

● ● ● CAPTURED OUTPUT

```
image: fedora43-cloud.qcow2
file format: qcow2
virtual size: 5 GiB (5368709120 bytes)
disk size: 556 MiB
cluster_size: 65536
Format specific information:
  compat: 1.1
  compression type: zlib
  lazy refcounts: false
  refcount bits: 16
```

6.2 Inspect with `virt-v2v-inspector` (dry-run)

`virt-v2v-inspector` does the full read-side of a conversion (mounts the disk, identifies the guest OS) but never writes output — perfect for previews.

● ● ● SHELL

```
distrobox enter fedora-virt -- \
  virt-v2v-inspector -i disk /var/tmp/fedora43-cloud.qcow2
```

Captured output (trimmed to the XML payload):

```
<?xml version='1.0' encoding='utf-8'?>
<v2v-inspection>
  <!-- generated by virt-v2v-inspector 2.10.0fedora=43,release=1.fc43 -->
  <program>virt-v2v-inspector</program>
  <package>virt-v2v</package>
  <version>2.10.0</version>
  <disks>
    <disk index='0'>
      <virtual-size>5368709120</virtual-size>
      <allocated estimated='true'>801964032</allocated>
    </disk>
  </disks>
  <firmware type='bios' />
  <operatingsystem>
    <name>linux</name>
    <distro>fedora</distro>
    <osinfo>fedora43</osinfo>
    <arch>x86_64</arch>
    <major_version>43</major_version>
    <minor_version>0</minor_version>
    <package_format>rpm</package_format>
    <package_management>dnf</package_management>
    <product_name>Fedora Linux 43 (Cloud Edition)</product_name>
    <product_variant>unknown</product_variant>
    <root>btrfsvol:/dev/sda4/root</root>
    <mountpoints>
      <mountpoint dev='btrfsvol:/dev/sda4/root'>/</mountpoint>
      <mountpoint dev='btrfsvol:/dev/sda4/var'>/var</mountpoint>
      <mountpoint dev='btrfsvol:/dev/sda4/home'>/home</mountpoint>
      <mountpoint dev='/dev/sda3'>/boot</mountpoint>
      <mountpoint dev='/dev/sda2'>/boot/efi</mountpoint>
    </mountpoints>
  </operatingsystem>
</v2v-inspection>
```

✓ Note `btrfsvol:/dev/sda4/root` — the inspector correctly detected the Btrfs subvolume layout shipped in the F43 cloud image.

6.3 Convert to a local libvirt directory

`-o local` (alias for `-o disk` with libvirt XML metadata) writes the converted disk plus a libvirt domain XML next to it.

```
SHELL BASH

distrobox enter fedora-virt -- bash -c '
  cd /var/tmp
  mkdir -p out
  virt-v2v -i disk fedora43-cloud.qcow2 \
    -o local -os ./out \
    -of qcow2 -on fedora43-converted
'
```

Captured run (progress bar collapsed):

```
CAPTURED OUTPUT · CONVERSION LOG

[  0.0] Setting up the source: -i disk fedora43-cloud.qcow2
[  1.0] Opening the source
[ 11.2] Checking filesystem integrity before conversion
[ 12.3] Detecting if this guest uses BIOS or UEFI to boot
[ 12.7] Inspecting the source
[ 13.9] Detecting the boot device
[ 13.9] Checking for sufficient free disk space in the guest
[ 13.9] Converting Fedora Linux 43 (Cloud Edition) (fedora43) to run on KVM
virt-v2v: This guest has virtio drivers installed.
[ 29.8] Setting a random seed
[ 29.8] SELinux relabelling
[ 31.2] Mapping filesystem data to avoid copying unused and blank areas
[ 32.4] Checking filesystem integrity after conversion
[ 33.4] Closing the overlay
[ 33.6] Assigning disks to buses
[ 33.6] Checking if the guest needs BIOS or UEFI to boot
[ 33.6] Setting up the destination: -o disk -os ./out
[ 34.6] Copying disk 1/1
      (... progress ...)
[ 36.2] Creating output metadata
[ 36.2] Finishing off
```

✓ Total wall-clock: **36 seconds**, single disk, no parallelism.

6.4 Inspect the output

● ● ● CAPTURED OUTPUT

```
$ distrobox enter fedora-virt -- ls -lh /var/tmp/out/
total 767M
-rw-r--r--. 1 starlight starlight 767M Jun  9 14:22 fedora43-converted-sda
-rw-r--r--. 1 starlight starlight 1.7K Jun  9 14:22 fedora43-converted.xml
```

The generated libvirt domain XML:

```
<?xml version='1.0' encoding='utf-8'?>
<domain type='kvm'>
  <!-- generated by virt-v2v 2.10.0fedora=43,release=1.fc43 -->
  <name>fedora43-converted</name>
  <metadata>
    <libosinfo:libosinfo
xmlns:libosinfo='http://libosinfo.org/xmlns/libvirt/domain/1.0'>
      <libosinfo:os id='http://fedoraproject.org/fedora/43' />
    </libosinfo:libosinfo>
  </metadata>
  <memory unit='KiB'>2097152</memory>
  <currentMemory unit='KiB'>2097152</currentMemory>
  <vcpu>1</vcpu>
  <cpu mode='host-model' />
  <features>
    <acpi/>
    <apic/>
  </features>
  <os>
    <type arch='x86_64' machine='q35'>hvm</type>
  </os>
  <clock offset='utc' />
  <on_poweroff>destroy</on_poweroff>
  <on_reboot>restart</on_reboot>
  <on_crash>restart</on_crash>
  <devices>
    <disk type='file' device='disk'>
      <driver name='qemu' type='qcow2' />
      <source file='/var/tmp/./out/fedora43-converted-sda' />
      <target dev='vda' bus='virtio' />
      <boot order='1' />
    </disk>
    <interface type='network'>
      <source network='default' />
      <model type='virtio' />
    </interface>
    <video>
      <model type='vga' vram='16384' heads='1' />
    </video>
    <graphics type='sdl' />
    <rng model='virtio'>
      <backend model='random'>/dev/urandom</backend>
    </rng>
  </devices>
</domain>
```

```

</rng>
<memballoon model='virtio' />
<panic model='isa'>
  <address type='isa' iobase='0x505' />
</panic>
<vsock model='virtio' />
<input type='tablet' bus='usb' />
<input type='mouse' bus='ps2' />
<console type='pty' />
<controller type='virtio-serial' model='virtio' />
<channel type='unix'>
  <target type='virtio' name='org.qemu.guest_agent.0' />
</channel>
</devices>
</domain>

```

i The disk is bound to `bus='virtio'` as `vda`, the NIC is `virtio`, plus `virtio-rng` / `virtio-balloon` / `virtio-serial` — that's the v2v *Converting ... to run on KVM* step at work.

6.5 Boot the result on Starlight

If `libvirt` is reachable from the distrobox (via the shared host socket), define and start:

● ● ● SHELL

BASH

```

distrobox enter fedora-virt -- bash -c '
  virsh -c qemu:///system define /var/tmp/out/fedora43-converted.xml
  virsh -c qemu:///system start fedora43-converted
'
```



Other recipes

Non-VMware sources, plus a few output variants. The VMware-specific recipes live in §5.

7.1 Raw / qcow2 disk → local libvirt directory

● ● ● SHELL

```
virt-v2v -i disk guest.img \  
        -o local -os /var/tmp/out \  
        -of qcow2 -on my-guest
```

7.2 OVA appliance → local libvirt

● ● ● SHELL

```
virt-v2v -i ova /path/to/appliance.ova \  
        -o local -os /var/tmp
```

7.3 Convert directly into a libvirt pool

● ● ● SHELL

```
virt-v2v -i disk guest.qcow2 \  
        -o libvirt -os default \  
        -on imported-guest
```

7.4 Just produce a bootable qemu command-line

● ● ● SHELL

```
virt-v2v -i disk guest.qcow2 -o qemu -os /var/tmp  
# Writes /var/tmp/<name>.sh that runs qemu-system-x86_64 directly.
```

7.5 In-place conversion (no copy)

● ● ● SHELL

```
virt-v2v-in-place -i disk guest.qcow2 # or  
virt-v2v --in-place -i libvirt my-stopped-guest
```

Use when you have already copied the disk to your target storage and just want the OS to be virtio-fied.



SECTION 08

Customisation at conversion time

08

`virt-v2v` re-exports the entire `virt-customize` flag set, so you can modify the guest as it lands:

● ● ● SHELL · CONVERT + CUSTOMISE

BASH

```
virt-v2v -i disk guest.qcow2 -o local -os /var/tmp \  
  --hostname converted-host \  
  --root-password password:redhat \  
  --install qemu-guest-agent,cloud-init \  
  --firstboot-command 'systemctl enable --now qemu-guest-agent' \  
  --upload ./company-ca.pem:/etc/pki/ca-trust/source/anchors/
```



Troubleshooting cheat-sheet

SYMPTOM	LIKELY CAUSE / FIX
<code>inspection could not detect ...</code> <code>operating system</code>	Source is a blank/unsupported disk (e.g. cirros/busybox). Verify with <code>virt-filesystems -a </code> and <code>guestfish --ro -a -i</code> .
<code>error: out of memory</code> during SELinux relabel	Bump appliance RAM: <code>-m 4096</code> .
Windows guest reboots forever on first boot	Disable Fast Startup / hibernation pre-conversion, ensure virtio-win ISO is reachable.
<code>vddk-libdir</code> missing for vCenter	Install VMware VDDK and point <code>-io vddk-libdir=</code> at the extracted distrib.
Conversion succeeds but VM has no network	Map source NIC into a present libvirt network: <code>--bridge sourceBridge:virbr0</code> or <code>--network vmnet1:default</code> .
Disk space exhausted in <code>/var/tmp</code>	Override scratch dir: <code>export VIRT_V2V_TMPDIR=/big/scratch</code> .
LUKS-encrypted root won't open	Pass keys: <code>--key /dev/sda3:key:s3cret</code> . TPM-sealed keys won't survive — re-seal post-conversion.



Pre / post-conversion checklist



Before

- ◆ Shut the guest down cleanly on the source hypervisor. Snapshots are fine; running VMs are not.
- ◆ **Windows:** disable BitLocker (or supply key), disable Fast Startup, loosen PowerShell `ExecutionPolicy` if firstboot scripts need to run, remove AV / Group Policy locks.
- ◆ **Linux:** make sure a real (non-Xen-PV) kernel is installed if the source is Xen PV.



After

- ◆ Boot the converted guest *with no console interruption* the first time — Windows will install virtio drivers and reboot several times.
- ◆ Remove VMware Tools manually if v2v's automatic removal failed.
- ◆ Re-IP the guest if the new bridge is on a different subnet.
- ◆ Re-attach data disks that were intentionally skipped during conversion.



References

- ◆ virt-v2v(1) man page — libguestfs.org/virt-v2v.1.html
- ◆ virt-v2v-inspector(1) — libguestfs.org/virt-v2v-inspector.1.html
- ◆ libguestfs project home — libguestfs.org
- ◆ Source: `setup-distrobox.sh --virt` module installs `virt-manager virt-viewer libvirt-client virt-v2v` from the Fedora 43 repos.